Deep Learning and Computation

The deep learning story — Lecture 18 (Chapter 10, Part 1)

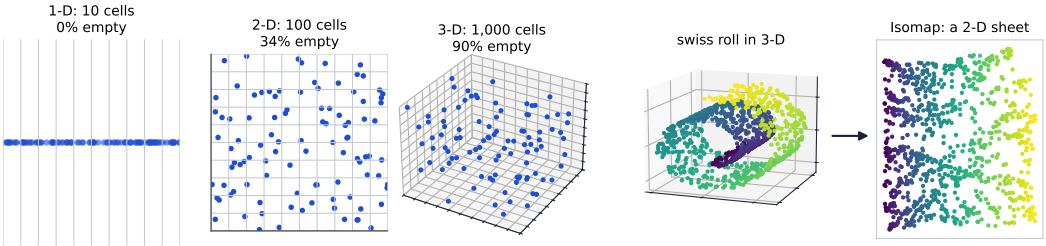
Amir Farbin

Part 1 — The deep learning story

Where we left off

- Classical machine learning learns **where to draw the boundary**, on features someone chose
- Deep learning learns **the features too**: from pixels, sound or particle hits to an answer
- Since 2012 it has taken over vision, speech, language, games, and much of physics
- Today: the story, in broad strokes. The pieces, one by one, come later

Natural data lives on manifolds



the same N = 100 points, 10 bins per axis

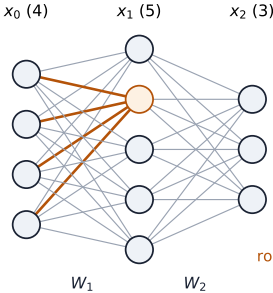
colour = position along the roll

- In many dimensions almost every box is empty: the **curse of dimensionality**
- But real data fills a thin, curved sheet inside that space, and its coordinates are the **features**

Why deep: a hierarchy of features

- Pixels $\rightarrow$ edges $\rightarrow$ parts $\rightarrow$ objects. Characters $\rightarrow$ words $\rightarrow$ sentences
- Each **layer** builds on the one below: low layers are shared by everything, high layers are specific
- A network with many layers can learn that hierarchy from examples

The building block



inputs (4 columns)

outputs (5 rows)

w_{11}	w_{12}	w_{13}	w_{14}
w_{21}	w_{22}	w_{23}	w_{24}
w_{31}	w_{32}	w_{33}	w_{34}
w_{41}	w_{42}	w_{43}	w_{44}
w_{51}	w_{52}	w_{53}	w_{54}

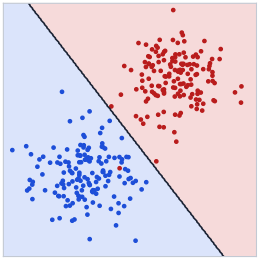
W_1: 5×4

row 2 of W_1: the weights into hidden unit 2

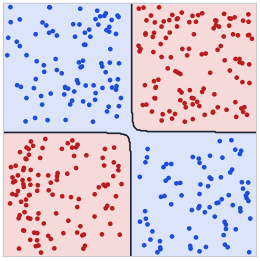
$$x_i = f(W_i x_{i-1} + b_i)$$

- A **layer**: $x_i = f(W_i x_{i-1} + b_i)$: a linear model, then a nonlinearity
- A **network**: layers stacked. Training finds every W and b at once

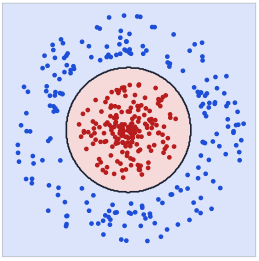
What a network can learn



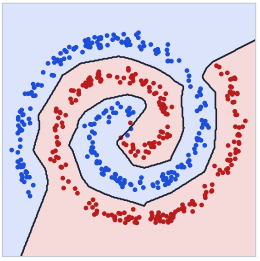
linear
inputs x_1, x_2
training accuracy 100%



XOR
add $x_1 x_2$
training accuracy 100%



circle
add $x_1^2 + x_2^2$
training accuracy 100%



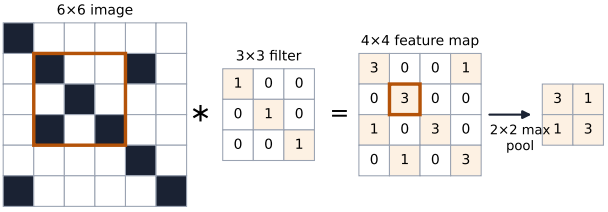
spiral
2 hidden layers of 16 (ReLU)
training accuracy 99%

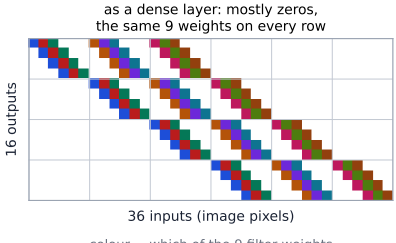
- With the right feature, a single neuron is enough. Without it, the network **finds** it
- On the TensorFlow playground you can watch this happen

Training, in one slide

- Pick the **output** to match the task (a probability, a class, a number) and a **cost** that scores it
- Walk **downhill** on the cost: $\theta \leftarrow \theta - \epsilon \nabla C$
- The framework computes the gradient for you (**automatic differentiation**), a mini-batch at a time
- Watch for **over-fitting**: keep data the network never trains on

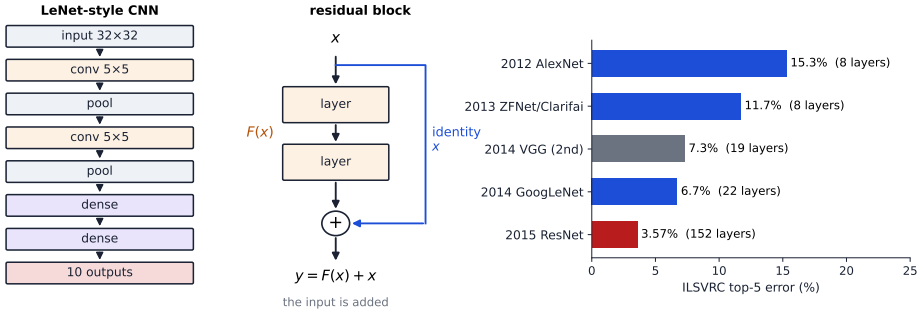
Images: learning the features





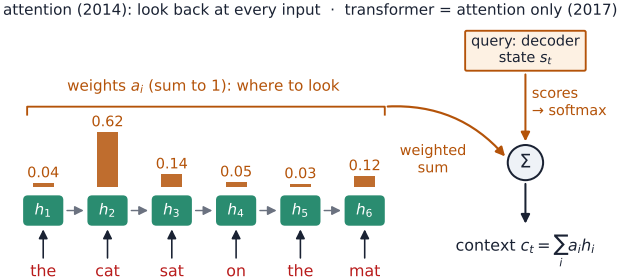
- A **convolution** slides a small filter over the image; the network **learns** the filters
- Stack them: edges, then parts, then objects. This is what won ImageNet in 2012

Bigger, deeper, and reusable



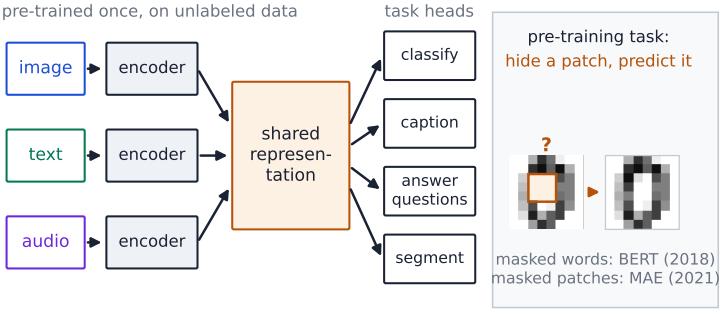
- From 8 layers (2012) to 152 (2015): **residual connections** made depth trainable
- What a network learns belongs to the **domain**: download one and **fine-tune** it for your task

Sequences and language



- **Recurrent** networks read one step at a time and keep a memory
- **Attention** (2014) looks back at everything; the **transformer** (2017) is attention only
- Large language models are transformers trained to **predict the next token**

Learning without labels



- **Autoencoders:** squeeze the input through a bottleneck and reproduce it
- **Foundation models:** pre-train on huge unlabeled data, then add a small head per task

Making new data



auto-regressive	**VAE**	**GAN**	**diffusion**	**normalizing flow**
one piece at a time, each given the ones before: $\prod_i p(x_i \mid x_{<i})$	encode to z, decode back; generate from $z \sim N(0, I)$	generator vs. discriminator: a two-player game	learn to remove noise step by step; start from pure noise	invertible map from $N(0, I)$; exact $p(x)$

- Learn the distribution of the data, then **sample** from it: images, text, simulations
- In physics: fast simulation learned from the full simulation

In my field

- 2014: deep networks on low-level detector data match decades of hand-built features (the SUSY dataset's own paper)
- Graph networks on calorimeters, transformers on sets of particles
- Autoencoders in the trigger, to keep events nobody thought to look for
- Networks trained to **ignore** what they shouldn't use, so they can't fake a discovery

What comes next

- **scikit-learn:** classical machine learning in a few lines (Lecture 18's notebook)
- **The software:** how frameworks compute and differentiate, and Keras (Lecture 20)
- **The pieces:** layers, activations, costs, optimizers, over-fitting, and the architectures